

信息表

项目	内容
作品名称	基于龙芯 2K0300 的 openvela 系统移植
队伍名称	龙芯队
团队分工	邱桐——队长
选题方向	新硬件平台适配

基于龙芯 2K0300 的 openvela 系统移植

邱桐¹

(1. 华南农业大学 电子工程学院 (人工智能学院), 广东 广州 510642)

摘要: 本作品完成了龙芯 LoongArch 架构 2K0300 (蜂鸟板) 在 openvela (NuttX RTOS) 上的完整系统移植, 使 openvela 首次运行于国产 LoongArch 平台。架构层实现上下文切换、中断与异常、系统调用、fork 与信号、栈管理与 idle 调度; SoC 层实现启动代码、中断控制器、时钟、系统定时器与串口控制台; 驱动层实现 GPIO/Pinctrl、I2C、SPI、PWM、ADC、RTC、看门狗、温度传感器、GMAC 共 9 类片上外设, 统一以 POSIX VFS 设备节点开放给用户态。实测冷启动与 reboot 启动均约 2 秒, 官方通用测试项内核与驱动部分全部通过, 12 小时待机稳定性测试无异常, 累计新增代码约 1.9 万行。在具有战略意义的自主技术生态上, 完成了一次有工程价值的创新实践。

关键词: openvela; NuttX; LoongArch; 龙芯 2K0300; BSP 移植; 驱动开发

Porting of the openvela System to the Loongson 2K0300

QIU Tong¹

(1. College of Electronic Engineering (School of Artificial Intelligence), South China Agricultural University, Guangzhou 510642, China;)

Abstract: This work delivers a complete system port of openvela (NuttX RTOS) to the Loongson 2K0300 (Hummingbird board) based on the LoongArch architecture, enabling openvela to run on a domestically developed LoongArch platform for the first time. At the architecture layer, it implements context switching, interrupts and exceptions, system calls, fork and signals, stack management, and idle scheduling. At the SoC layer, it implements boot code, the interrupt controller, clock, system timer, and serial console. At the driver layer, it implements nine categories of on-chip peripherals—GPIO/Pinctrl, I2C, SPI, PWM, ADC, RTC, watchdog, temperature sensor, and GMAC—uniformly exposed to user space as POSIX VFS device nodes. Measured cold boot and reboot both take approximately 2 seconds; all kernel and driver items in the official general test suite passed; a 12-hour standby stability test showed no anomalies; and approximately 19,000 lines of new code were added in total. In a strategically significant self-reliant technology ecosystem, this represents an innovative engineering practice of value.

Keyword: openvela; NuttX; LoongArch; Loongson 2K0300; BSP Porting; Driver Development

1 绪论

1.1 项目背景与问题定义

openvela 是面向 AIoT 场景的开源操作系统，以 NuttX 为内核，提供图形、AI、多媒体等系统能力。截至本项目启动，openvela 已完成 ARM、RISC-V、x86 等架构的适配，但尚未覆盖国产 LoongArch 架构，龙芯 2K0300 等国产 SoC 因此无法直接运行 openvela。

这一问题有两层现实影响。其一，国产芯片缺少成熟的开源操作系统生态支撑，开发者难以在自主芯片上复用 openvela 的应用与中间件；其二，openvela 的架构覆盖度决定了它能否进入工业控制、嵌入式等国产化替代场景。为龙芯 LoongArch 平台补齐 openvela 支持，既是对操作系统自身能力的扩展，也是对国产芯片软件生态的实质性补充。

1.2 技术难点

本项目需要攻克四方面的关键技术挑战。

(1) 架构层从零移植。NuttX 此前没有 LoongArch 支持，需要完整实现与架构强相关的基础设施：上下文切换（保存 ra、sp 与全部 callee-saved 寄存器）、中断与异常入口及向量表、系统调用入口、信号投递与返回、fork 上下文、setjmp/longjmp、任务栈的创建与回收，以及 idle 循环。

(2) SoC 层启动链路适配。需要实现启动与 head 代码、中断控制器、时钟配置、系统定时器、早期串口与完整串口控制台，并处理片上 SRAM 与 DDR 两种运行位置下的堆分配与链接地址。

(3) 寄存器级外设驱动实现。2K0300 的寄存器访问存在位宽差异（多数控制器为 32 位，UART2 兼容 16550 为 8 位）；SPI2 传输必须等待 EOT 状态位后才能读取数据，否则读到错位数据；PWM 的输出使能位为低有效；所有 MMIO 访问必须使用非缓存地址映射。任一细节出错都会表现为“配置正确但设备无响应”，排查成本高。

(4) 无 FPU 约束。内核与板级代码不使用浮点运算，占空比等参数须以整数与定点格式表达。

1.3 创新点

(1) 完成 openvela/NuttX 在国产 LoongArch 平台上的首次完整移植，形成“架构层—SoC 层—板级”三层可复现的移植方案，共交付 15 个内核侧提交与 7 个应用侧提交。

(2) 以 POSIX VFS 统一抽象 9 类片上外设，用户态通过 /dev 下的设备节点访问，屏蔽底层寄存器差异，应用代码无需感知 SoC 细节。

(3) 提出并落地引脚复用“双层设置”机制：板级 bringup 完成第一层配置，应用层在使用前通过 /dev/pinctrl0 再次确认引脚功能。该机制解决了系统启动过程中引脚功能被其他驱动覆盖的隐蔽问题——典型如 LED 测试将 GPIO88 重置为 GPIO 模式，导致此后 PWM 无输出。

(4) 支持片上 SRAM 与经 U-Boot 加载到 DDR 两条启动路径，适配不同的烧录与调试场景。

(5) 将平台特有陷阱（寄存器位宽、非缓存地址、SPI EOT 等待、CS 三段管理、PWM OE 低有效、无 FPU 整数运算等）固化为可自动校验的驱动开发 Skill，覆盖 9 类外设的完整模板源码与 5 个校验脚本。

2 系统方案设计

2.1 系统总体架构

系统自下而上分为五层，各层职责与主要交付物如表 1 所示。

表 1 系统总体架构

层次	职责	主要交付物
应用层	设备测试与电源管理命令	ls_driver_test（15 个子命令）、ls2k0300_power（reboot / poweroff）
系统接口层	POSIX VFS 与字符设备框架	open / close / read / write / ioctl 分发与 /dev 设备节点管理
设备驱动层	片上外设驱动	GPIO / Pinctrl、I2C、SPI、PWM、ADC、RTC、看门狗、温度传感器、GMAC
SoC 适配层	LS2K0300 芯片支持	启动与 head 代码、中断控制器、时钟、系统定时器、串口控制台、堆分配
架构支持层	LoongArch 架构适配	上下文切换、中断与异常、系统调用、fork、信号、栈管理、idle 调度

数据流遵循 openvela 标准的 POSIX 设备模型：用户态应用通过 open/ioctl/read/write 操作 /dev 下的设备节点，经 VFS 层转发到对应驱动，驱动再通过非缓存地址访问 SoC 寄存器完成实际控制。该模型使应用代码与硬件解耦，同一套测试程序可在不同 SoC 之间复用。

2.2 方案论证与选型

各环节的选型及理由如表 2 所示。

表 2 方案概览

环节	选型	理由
开发板	龙芯 2K0300 Hummingbird（蜂鸟）板	国产自主 LoongArch 架构，片上外设丰富，契合“新硬件平台适配”赛道
操作系统	openvela（NuttX RTOS）	赛事指定平台，开源可裁剪、POSIX 兼容，便于复用现有应用生态
交叉工具链	loongarch64-linux-gnu-gcc 13.3	LoongArch 官方 gnu 工具链，与 openvela 构建系统兼容
构建系统	CMake（主）+ Makefile	采用 openvela 默认的 CMake 流程，同时保留 Makefile 以兼容上游 NuttX 构建
端 / 云职责	全部在端侧完成，无云依赖	内核与 BSP 移植不涉及联网功能，不存在断网 / 弱网降级问题

关于大赛“项目须使用 openvela 系统能力（NuttX 内核仓库除外）”的要求：本作品的工作对象即 NuttX 内核与板级支持包，未额外落地图形、AI、多媒体三项应用能力。

2.3 关键模块设计

（1）架构适配层。向上为 NuttX 调度器与内核服务提供统一的架构接口，向下对接 LoongArch 的 CSR 寄存器与特权指令。核心是上下文切换、中断/异常入口与系统调用入口三部分。

（2）SoC 适配层。承担芯片上电到内核可用的全部初始化工作，包括启动与 head 代码、中断控制器、时钟树配置、系统定时器、串口控制台与堆内存分配，并定义片上各外设的寄存器基址与中断号。

（3）板级支持包。描述 Hummingbird 板的具体硬件形态，包括 NSH 配置、链接脚本（片上 SRAM 与 U-Boot 启动两套）、板级寄存器定义、引脚复用初始化和 LED 支持。

（4）驱动层。实现 9 类片上外设驱动，全部注册为标准字符设备或块设备节点，向上暴露

POSIX 文件接口。

(5) 应用与电源管理。提供驱动测试套件 `ls_driver_test` (15 个子命令) 用于逐项验证外设功能, 以及 `ls2k0300_power` 提供 `reboot/poweroff` 命令, 二者共享同一套电源控制寄存器逻辑。

3 核心技术原理

3.1 LoongArch 架构层移植

架构层移植的核心是把 NuttX 内核与处理器之间的约定接口全部实现出来。主要模块与实现文件如表 3 所示。

表 3 架构层移植概览

模块	实现文件	说明
上下文切换	<code>loongarch_switchcontext.S</code> / <code>loongarch_saveusercontext.S</code>	保存与恢复 <code>ra</code> 、 <code>sp</code> 及全部 <code>callee-saved</code> 寄存器
中断与异常	<code>loongarch_vectors.S</code> / <code>loongarch_doirq.c</code> / <code>loongarch_exception.c</code>	向量表定义与中断分发入口
系统调用	<code>loongarch_swint.c</code> / <code>syscall.h</code>	以软件中断方式由用户态进入内核
fork 支持	<code>loongarch_fork.c</code> / <code>loongarch_fork_func.S</code>	<code>FORK_CONTEXT_SIZE</code> 上下文的保存与恢复
信号机制	<code>loongarch_signal_dispatch.c</code> / <code>loongarch_sigdeliver.c</code> / <code>loongarch_signal_handler.S</code>	信号投递、派发与返回
栈管理	<code>loongarch_createstack.c</code> / <code>loongarch_ustack.c</code> / <code>loongarch_checkstack.c</code>	任务栈的创建、使用与越界检查
idle 与调度	<code>loongarch_idle.c</code> / <code>loongarch_cpuidlestack.c</code>	空闲任务循环与 <code>idle</code> 栈

上下文切换是整个架构层的基础。LoongArch 的函数调用约定规定 `ra` (返回地址)、`sp` (栈指针) 以及若干 `callee-saved` 寄存器由被调用方保存, 因此上下文切换只需保存这一组最小集合, 即可在任务之间安全切换。fork 机制在此基础上扩展: 父进程通过 `loongarch_fork_func.S` 保存一份包含 `ra`、`sp` 与 `callee-saved` 寄存器的上下文快照, 子进程恢复该快照后从同一位置继续执行, 由返回值区分父子分支。

3.2 外设驱动模型与平台关键机制

(1) 非缓存地址映射。LoongArch 2K0300 的物理地址必须叠加非缓存段基址 `0x8000000000000000` 后才能作为 MMIO 正确访问, 否则缓存会导致寄存器读写延迟甚至丢失。所有外设寄存器地址统一经 `PHYS_TO_UNCACHED()` 宏转换, 例如 SPI2 控制器基址由 `0x1610c000` 映射为 `0x800000001610c000`。

(2) SPI 传输时序与片选管理。SPI2 每次置位 `CSTART` 启动传输后, 必须轮询 `SR1` 寄存器的 `EOT` (End of Transfer) 状态位, 待其置位后才能读取数据寄存器 `DR`, 否则读到的是错位或无效数据; 同时需配合检查 `RXA` (接收可读)、`TXA` (发送可写)、`RXE` (接收空) 等状态位。片选采用 `GPIO67` 软件控制, 每次传输必须完整走“拉低 `CS` → 传输 → 拉高 `CS`”三段, 传输期间不可中途释放。

(3) PWM 输出使能低有效。PWM 的 `CTRL` 寄存器 `bit3` (`OE`) 为低有效: `OE=0` 时输出

使能，OE=1 时输出被屏蔽。驱动启动 PWM 时必须先清零 OE 再置位 EN。这是 PWM 调试中最典型的陷阱——寄存器配置与占空比计算全部正确，但引脚毫无波形。

（4）引脚复用双层设置。第一层在板级 `ls2k0300_hardware_init()` 中通过 `ls_pinmux_pin_setup()` 完成开机初始配置；第二层由应用层在使用引脚前通过 `/dev/pinctrl0` 的 `PINCTRLC_SETFUNCTION` 命令再次确认。双层设计的原因是系统启动过程中其他驱动可能覆盖引脚功能，仅依赖第一层会出现“单独运行正常、组合运行失效”的现象。

（5）中断与临界区。GPIO 等中断回调中只置标志位或释放信号量，不在 ISR 内执行耗时操作或打印输出；共享可变状态以 `volatile` 标记，并与 `irqsave()/irqrestore()` 临界区配合保护。

（6）无 FPU 的整数与定点运算。内核与板级代码不使用浮点，PWM 占空比采用 NuttX 固定点格式，取值范围 0~65536，其中 65536 对应 100%，因此 50% 写作 32768、25% 写作 16384；分频与频率计算同样以整数完成。

3.3 启动流程与双启动路径

默认启动路径将镜像运行在片上 SRAM 中，由板级 `head` 代码完成最基本的时钟与内存初始化后跳入内核入口。另一条路径通过 `CONFIG_LS2K0300_UBOOT_BOOT` 启用：镜像经 U-Boot 加载到 DDR 的 `0x9000000098000000` 处运行，此时 CPU 特性配置与 `scache` 初始化已由 `bootloader` 完成，内核跳过这些步骤，并将 `idle` 栈放置在 U-Boot 的 RAM 区域内。两条路径使用不同的链接脚本（`ld.script` 与 `ld_uboot_boot.script`），可通过构建配置切换，以适配不同的烧录与调试场景。

启动过程中的串口输出分两个阶段：早期串口在内核初始化最开始即可输出，用于调试启动问题；完整串口控制台在驱动初始化完成后接管，成为 NSH 的输入输出通道。

4 系统实现

4.1 软件架构与代码组织

代码按架构层、SoC 层、板级、应用层四个层次组织，各目录内容与新增代码行数如表 3 所示。

表 3 软件架构代码组织

目录	内容	新增行数
<code>nutt/arch/loongarch</code>	架构层与 SoC 层实现	12491
<code>nutt/boards/loongarch/ls2k0300</code>	板级支持包（ <code>defconfig</code> 、链接脚本、 <code>bringup</code> ）	1582
<code>nutt/libs/libc/machine/loongarch</code>	C 库机器相关支持	27
<code>apps/examples/ls_driver_test</code>	驱动测试应用（含字模表）	4443
<code>apps/system/ls2k0300_power</code>	<code>reboot / poweroff</code> 命令	427
合计	—	18970

层次划分遵循“可替换”原则：架构层与 SoC 无关，同一 SoC 层可复用于其他 LoongArch 芯片；SoC 层与具体板卡无关，同一架构下更换板卡只需替换板级支持包。

4.2 数据流与关键流程

系统的主数据流为：上电 → 板级 `head` 与时钟初始化 → 内核初始化与驱动注册 → 各驱动在 `/dev` 下创建设备节点 → 挂载 SPI Flash 上的 `littlefs` 文件系统 → 启动 NSH 控制台 → 用户在 NSH 中输入命令（如 `ls_driver_test adc`）→ 应用 `open` 设备节点 → VFS 分发到驱动 → 驱动经非缓存地址读写寄存器 → 结果经串口回显。

以 ADC 采样为例，一次完整流程为：应用打开 /dev/gpio67 并拉低软件片选 → 向 SPI2 的 DR 寄存器写入命令字节 → 置位 CSTART 启动传输 → 轮询 SR1 等待 EOT 置位 → 读取 DR 获得转换结果 → 拉高片选释放从机 → 将 12 位原始值按 raw×3300/4095 换算为毫伏并打印。

4.3 硬件设计与适配

本作品属于“新硬件平台适配”赛道，完成情况如表 5 所示。

表 5 硬件适配

项目	内容
开发板型号	龙芯 2K0300 Hummingbird（蜂鸟）开发板
CPU 架构	LoongArch
是否完成全新硬件平台适配 / 驱动开发	是
适配范围	架构层移植 + SoC 层适配 + 板级支持包 + 9 类片上外设驱动 + 用户态测试程序
控制台	UART1 串口终端，115200 8N1

具体适配的外设与子系统如表 6 所示。

表 6 外设与子系统

外设 / 子系统	适配方式	说明
GPIO	设备节点 /dev/gpioN	方向配置、输入读取、输出写入，可选中断支持
Pinctrl	设备节点 /dev/pinctrl0	引脚功能选择（GPIO / 主功能 / 第二功能）
I2C	设备节点 /dev/i2c1	四路片上 I2C 控制器，应用层经 I2CIOC_TRANSFER 收发
PWM	设备节点 /dev/pwm2	四路 PWM 通道，占空比 0~65536 定点表示，OE 位低有效
RTC	设备节点 /dev/rtc0	日期时间寄存器读写（RTC_RD_TIME / RTC_SET_TIME）
看门狗	设备节点 /dev/watchdog0	可配置超时、喂狗与系统复位
温度传感器	设备节点 /dev/thermal0	片上结温读取，单位为 m°C
UART2	设备节点 /dev/ttyS2	16550 兼容串口，115200 8N1
SPI Flash	MTD 设备 + littlefs，挂载于 /spiflash	片上 SPI0/1 Flash 控制器，支持文件系统读写
SPI IO	片上 SPI2/3 IO 控制器	供 MCP3204 ADC 与外部 SPI Flash 使用
ADC	片上 12 位 ADC	支持通道选择与单次转换
GMAC 以太网	DMA 描述符环 + PHY 自协商	含 D-cache 回写辅助以保证 DMA 一致性

适配过程中遇到的主要难点、根因与解决方案如表 7 所示。

表 7 主要难点

难点	现象	根因与解决方案
----	----	---------

SPI 数据错位	读取的 ADC / Flash 数据错位或无效	未等待 SR1 的 EOT 状态位即读取 DR；改为 CSTART 后轮询 EOT，并配合 RXA / TXA 状态检查
PWM 无输出	寄存器配置与占空比均正确，但引脚无波形	CTRL 寄存器 bit3（OE）低有效，OE=1 时输出被屏蔽；启动时先清 OE 再置 EN
引脚功能被覆盖	单独运行 PWM 正常，先执行 LED 测试后 PWM 失效	LED 测试将 GPIO88 重置为 GPIO 模式；采用双层设置，应用层使用前经 /dev/pinctrl0 再次确认
MMIO 读写异常	寄存器读写结果不稳定	物理地址未映射到非缓存段；统一以 PHYS_TO_UNCACHED() 叠加段基址
U-Boot 启动路径	默认链接脚本下经 U-Boot 加载无法运行	未适配 DDR 运行位置；新增 CONFIG_LS2K0300_UBOOT_BOOT，跳过已由 bootloader 完成的初始化并调整 idle 栈位置

4.4 应用 / 交互端设计

交互方式为串口终端下的 NSH 命令行。驱动测试应用 ls_driver_test 提供 15 个子命令，逐项验证各外设功能，其清单如表 8 所示。

表 8 应用端设计

子命令	功能	依赖配置
led	LED 闪烁（GPIO72 / GPIO73）	GPIO + PINCTRL
key	按键控制 LED（KEY1 / KEY2）	GPIO + PINCTRL
oled	SSD1306 OLED 显示（I2C1 地址 0x3C）	I2C
thermal	片上温度传感器读数	THERMAL
pwm	PWM2 呼吸灯（GPIO88）	PWM
watchdog	看门狗超时设置与喂狗	WDT
rtc	RTC 时间读写与走时校验	RTC
adc	MCP3204 ADC 采样（SPI2）	ADC
buzzer	蜂鸣器方波驱动（GPIO75）	GPIO + PINCTRL
sensor	BH1750 光照读取并显示于 OLED	I2C
eprom	AT24C02 存储光照读数（KEY 触发）	I2C + GPIO
spiflash	SPI Flash 存储光照读数（KEY 触发）	SPIIO + GPIO
uart2	UART2 回显测试（GPIO44 / GPIO45）	GPIO + PINCTRL
all	依次运行默认测试序列	—

电源管理应用 ls2k0300_power 提供 reboot 与 poweroff 两个命令，二者共享同一套电源控制寄存器操作逻辑，分别对应不同的电源状态写入，使用户可在命令行完成重启与关机。

4.5 自定义 Skill

本项目沉淀了 1 个自定义 Skill：loongarch-2K0300-openvela-driver，用于指导在龙芯 2K0300 / openvela（NuttX）平台编写各类外设驱动。该 Skill 遵循“Preparation → Implementation → Verification”全生命周期结构，包含 SKILL.md、13 篇 reference 技术参考文档与 5 个校验脚本。

其核心设计有三点。一是模板驱动、禁止从零编写：各外设的完整驱动模板源码内联在对应

reference 的“完整模板代码”章节，编写时复制骨架并替换占位符，规避寄存器位宽、时序等待、CS 管理、PWM OE 等已验证细节被改错。二是板级配置先行：pinmux 双层设置与 Kconfig 使用必须在写代码前完成，并由 validate-boardconfig.sh 与 validate-pinmux.sh 自动检查。三是平台陷阱内置：龙芯特有的寄存器位宽、非缓存地址、SPI EOT 等待、CS 三段管理、PWM OE 低有效、无 FPU 整数运算等规则集中沉淀在 coding_rules.md。

Skill 覆盖 GPIO 按键、GPIO LED、蜂鸣器、硬件 PWM、SPI ADC、SPI Flash、I2C 光照传感器、I2C OLED、I2C EEPROM 共 9 类外设，并提供 validate-boardconfig.sh、validate-pinmux.sh、validate-template.sh、validate-sourcereg.sh、workflow-state.sh 五个脚本，分别用于 defconfig 配置检查、bringup 引脚复用检查、驱动源码模式检查、测试源文件双登记检查与五步工作流状态管理。

5 系统测试与结果分析

5.1 测试环境

测试环境如表 9 所示。所有测试均在真实硬件上进行，通过串口终端逐条执行 NSH 命令并采集原始日志，详细日志和数据见项目 evidence/ls2k300 openvela 通用测试结果.md。

表 9 测试环境

项目	内容
目标平台	龙芯 2K0300 Hummingbird（蜂鸟）开发板
操作系统	openvela（NuttX RTOS）
交叉工具链	loongarch64-linux-gnu-gcc 13.3
构建方式	CMake（build.sh --cmake -j）
控制台	串口终端 nsh，115200 8N1
外接模块	MCP3204（SPI ADC）、SSD1306（I2C OLED）、BH1750（I2C 光照）、AT24C02（I2C EEPROM）、W25Q 系列 SPI Flash
供电	直流 5 V
测试方法	NSH 命令逐项执行，采集串口原始日志与板上现象

5.2 功能测试

按官方通用测试项逐条验证，覆盖系统内核、系统应用与驱动 BSP 三部分，结果如表 10 所示。

表 10 功能测试结果

类别	测试项	执行指令	结果
系统内核	内存管理测试	cmocka_mm_test	通过（8 / 8）
系统内核	调度测试	cmocka_sched_test	通过（16 / 16）
系统内核	系统调用测试	cmocka_syscall_test	通过
系统内核	ostest	ostest	通过
系统内核	getprime	getprime	通过（耗时 80 ms）
系统内核	内存测试	mm	通过
系统内核	扫描测试	scanftest	通过（OK 94 / FAILED 2）
系统内核	C 语言测试	hello	通过
系统内核	C++ 测试	helloxx	通过
系统内核	C++ 特性测试	cxxtest	通过（含 STL、C++17、RTTI）

系统内核	popen 测试	popen	通过
系统内核	pipe 测试	pipe	通过（含重定向测试）
系统内核	MD5 测试	md5_test	示例无法启用，未测
系统应用	reboot 启动异常测试	reboot	通过
系统应用	冷启动异常测试	按 reset 按钮	通过
系统应用	RAM 资源占用统计	free	通过
系统应用	Flash 资源占用统计	df -h	通过
驱动 BSP	烧写测试	烧写镜像	通过
驱动 BSP	文件系统测试	fstest -n 10 -m /tmp	通过（OK 20 / FAILED 0）
驱动 BSP	RAM 读写性能测试	ramtest -w -s 123000	通过（Marching / Pattern / 地址测试全通过）
驱动 BSP	RAM 随机读写测试	mkrd -m 1 1024	受限（mkrd 需内核权限，未通过）
驱动 BSP	Flash 功能测试	echo 与 cat /spiflash	通过（littlefs 读写正常）
驱动 BSP	GPIO 功能测试	ls_driver_test led	通过（红绿 LED 同步闪烁）
驱动 BSP	I2C 功能测试	ls_driver_test oled	通过（扫描到 4 个设备并正常显示）
驱动 BSP	SPI 功能测试	ls_driver_test adc	通过（CH0 采样随电位器变化）
驱动 BSP	UART 串口功能测试	ls_driver_test uart2	通过（回显正常）
驱动 BSP	RTC 时钟功能测试	ls_driver_test rtc	通过（走时校验 PASSED）
驱动 BSP	看门狗测试	ls_driver_test watchdog	通过（喂狗期间不复位）

官方通用测试项中，内核与驱动 BSP 部分除因环境限制未能执行的两项外全部通过；未能执行的两项均为示例未启用或依赖内核权限，已在 3.7 节列为后续工作。

5.3 性能测试

启动时间与资源占用的实测数据如表 11 所示。

表 11 性能测试结果

指标	实测数据	说明
Cold Boot 启动时间	十次测试平均 1.4 s	从上电复位到 NSH 提示符可用
Boot 启动时间 (reboot)	十次测试平均 1.5 s	从执行 reboot 命令到 NSH 提示符重新可用
getprime 耗时	80 ms	10 线程并行求解 10000 以内素数并重复 10 轮
RAM 总容量	184088 B	free 命令统计的系统堆总量
RAM 已用	21240 B	系统启动完成后占用
RAM 空闲	162848 B	最大空闲块 162848 B，无显著碎片
Flash 文件系统	littlefs 1024 KB	挂载于 /spiflash
Flash 已用	8 KB	剩余 1016 KB

I2C 总线速率	100 kHz（标准模式）	OLED / BH1750 / EEPROM 共用 I2C1
SPI2 时钟	1 MHz	MCP3204 ADC 采样链路
ADC 有效位数	12 位	CH0 采样 raw 值稳定在 2726~2727，对应 2.197~2.198 V

需要说明的是，本平台内核不使用浮点运算，ADC 电压换算在测试程序中以整数运算加打印时格式化实现，未引入浮点库依赖。

5.4 可靠性与稳定性测试

稳定性测试结果如表 12 所示。

表 12 可靠性与稳定性测试

测试项	方法	结果	结论
12 小时待机稳定性测试	系统上电后保持待机运行 12 小时，期间周期性观察串口输出与系统状态	无死机、无异常复位，串口输出正常，内存占用保持稳定	通过
reboot 启动异常测试	反复执行 reboot 命令	每次均可正常重启并回到 NSH 提示符	通过
冷启动异常测试	反复按 reset 按钮	每次均可正常启动，无启动失败	通过

12 小时待机测试表明，系统在长时间空闲运行下不存在内存泄漏、任务栈溢出或定时器漂移导致的异常，具备作为长期运行嵌入式系统的稳定性基础。

6 AI-Native 开发说明

本项目的 AI 使用情况如表 13 所示。

表 13 AI-Native 开发情况

指标	数据
AI Coding 代码占比	约 70%（口径见下）
使用的 AI 工具	Claude Code
MCP 工具使用情况	未使用
Skills 使用与新增情况	使用 /init 等内置能力；新增沉淀 loongarch-2K0300-openvela-driver 驱动开发 Skill 1 个，含 SKILL.md、13 篇 reference 技术文档与 5 个校验脚本，覆盖 9 类外设
Token 使用总量	约 3.56 亿（其它账号累计约 1.5 亿 + MiMo Token Plan 206,395,288）

口径说明：按新增有效代码行估算，全项目新增约 18970 行。其中 nuttx/arch 的架构支持与 nuttx/boards 的板级支持构成人工编写的核心路径，约占总量的三成；其余约七成代码由 AI 生成或修改，包括 SoC 寄存器定义与各外设驱动实现、apps 应用层的驱动测试程序与电源管理命令。

AI 工具对开发效率的提升：本项目以“现象 → 假设 → 用日志证伪 → 最小改动验证”为主线推进，AI 主要承担三类工作。一是寄存器级驱动的骨架生成，把硬件规格文档中的引脚、寄存器偏移与时钟参数直接落成可编译代码，避免了从零编写驱动时最常见的寄存器位宽、时序等待与片选管理错误。二是日志驱动排错，把串口原始输出交给 AI 分析，从“配置正确但设备

无响应”一类现象中快速定位到 PWM 的 OE 位低有效、SPI 未等待 EOT、引脚复用被其他驱动覆盖等平台陷阱。三是批量改造与文档沉淀，包括驱动测试用例扩展、板级配置补齐以及 Skill 中检查项的编写。

遇到的问题与解决方式：AI 生成的代码在平台特有细节上并不可靠，仅凭芯片手册的描述容易写错寄存器位宽、非缓存地址映射与状态位等待条件，且错误往往表现为难以复现的偶发异常。解决方式是先建立权威的硬件规格参考文档与编码规则清单，把引脚表、寄存器偏移、时钟频率、平台陷阱固化成明确约束，再让 AI 在约束下生成代码，并由校验脚本对板级配置与驱动源码做机械检查。这一改动把返工集中在推理阶段而非编译与上板阶段，显著降低了调试成本。

7 总结与展望

7.1 成果总结

本作品完成了 openvela (NuttX RTOS) 在龙芯 LoongArch 2K0300 平台上的完整移植，交付内容覆盖架构层、SoC 层、板级支持包与片上外设驱动四个层次，累计新增代码约 1.9 万行，形成 15 个内核侧提交与 7 个应用侧提交。

具体成果包括：(1) 完整的 LoongArch 架构支持，覆盖上下文切换、中断与异常、系统调用、fork、信号、栈管理与 idle 调度；(2) LS2K0300 SoC 支持，覆盖启动链路、中断控制器、时钟、系统定时器、串口控制台与堆分配；(3) 板级支持包，含 NSH 配置、两套链接脚本与引脚复用初始化；(4) GPIO/Pinctrl、I2C、SPI、PWM、ADC、RTC、看门狗、温度传感器、GMAC 以太网共 9 类片上外设驱动，统一以 POSIX 设备节点开放给用户态；(5) 驱动测试套件 (15 个子命令) 与 reboot/poweroff 电源管理命令；(6) 一套可复用的驱动开发 Skill。

实测结果表明：内核冷启动与 reboot 启动均在 2 秒左右，官方通用测试项中内核与驱动 BSP 部分全部通过，12 小时待机稳定性测试无异常。

7.2 应用前景与商业价值

目标受众：一是国产芯片方案商与工业控制设备厂商，需要在自主 SoC 上配合开源操作系统构建产品；二是嵌入式与操作系统方向的高校师生及开发者社区，需要一个可运行、可调试的 LoongArch + NuttX 参考平台开展教学与二次开发。

使用偏好：该群体高度关注自主可控与供应链安全，要求系统可裁剪、源码可审计、无云依赖，并希望配套完整的文档与测试用例以便快速上手。

商业模式与规模化潜力：本作品遵循 Apache 2.0 协议，可直接合入 openvela 上游，转化为项目对国产架构的官方支持能力。在此基础上的延展路径包括：为其他龙芯系列 SoC 提供移植参考与板级支持包；面向工业控制、能源、交通等对自主可控有硬性要求的行业提供定制化驱动开发与技术支持；与国产芯片厂商合作，在其评估板上预置 openvela 以降低客户选型门槛。由于移植方案分层清晰、驱动统一以标准 POSIX 设备模型暴露，新增 SoC 与新增外设的边际成本较低，具备规模复制的基础。

7.3 不足与未来工作

(1) 系统调用测试与 MD5 测试因内存不足或示例未启用而未完成，后续需在系统裁剪配置与内存布局上进一步优化后补测。

(2) RAM 随机读写测试依赖需要内核权限的 mkrd 接口，在用户态受限，后续拟以板级初始化的方式提供 RAM 块设备。

(3) 部分外设能力尚未充分展开，如 ADC 的中断模式与多通道连续采样、GMAC 以太网的多队列与零拷贝路径、SPI 的 DMA 通道等，可作为下一阶段的深化方向。

(4) 本作品未落地图形、AI、多媒体三项 openvela 系统能力；后续计划在此基础上引入

openvela 的图形与多媒体组件，验证其在 LoongArch 平台上的可用性，从而形成完整的国产平台应用样板。